

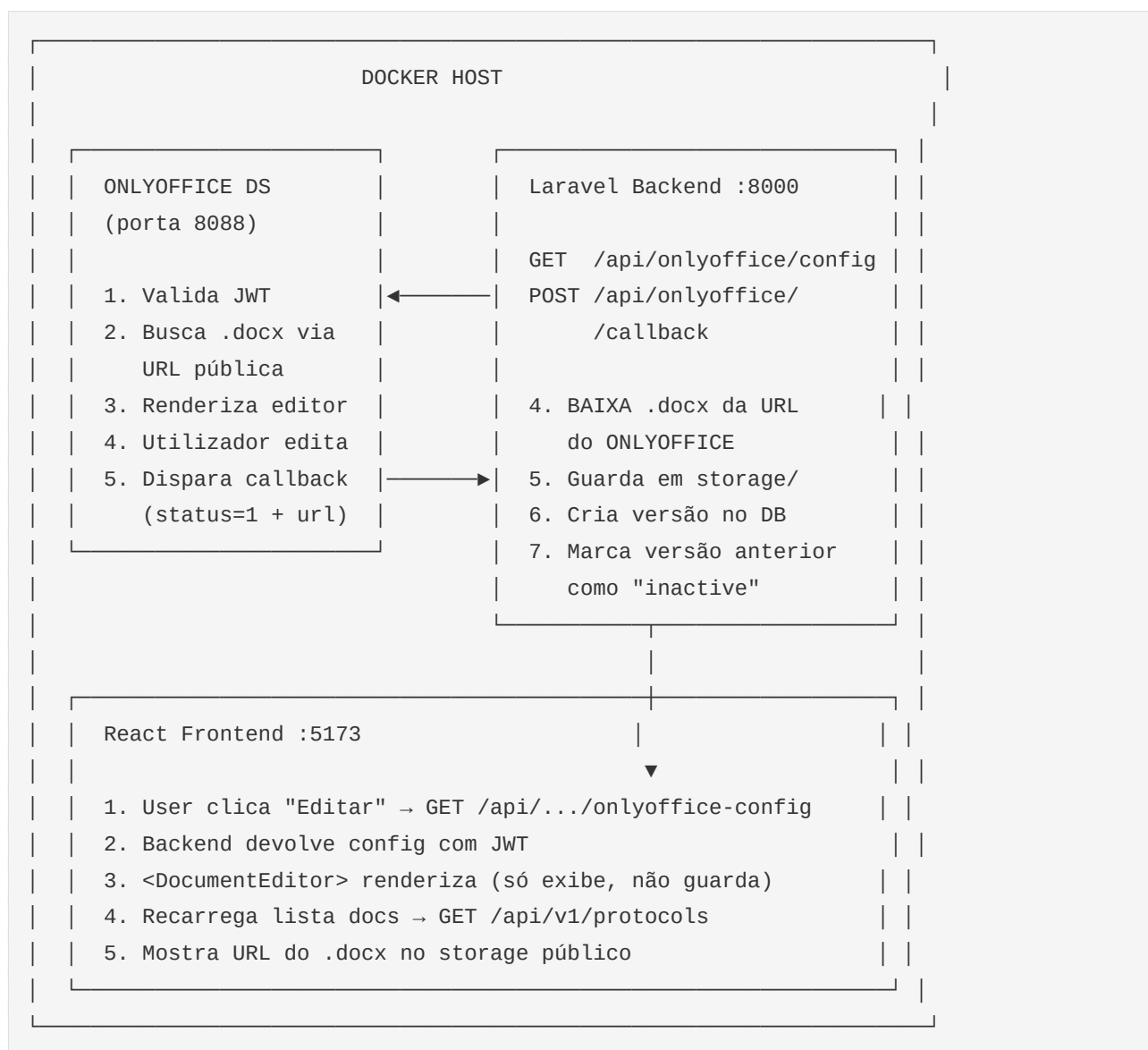
Plano de Integração ONLYOFFICE — SGPC-ISCISA

Sistema de Gestão de Protocolos e Monografias Científicas

Índice

1. [Visão Geral da Arquitectura](#)
2. [Estado Actual](#)
3. [Fluxo Correcto \(Backend Baixa o Documento\)](#)
4. [Ficheiros a Alterar / Criar](#)
5. [Fase 1 — Configuração e Ambiente](#)
6. [Fase 2 — Callback Real](#)
7. [Fase 3 — Config Endpoint para Documentos Reais](#)
8. [Fase 4 — Nova Rota Autenticada](#)
9. [Fase 5 — Componente Editor React](#)
10. [Fase 6 — Serviço Frontend Actualizado](#)
11. [Fase 7 — Integrar Editor nas Páginas de Protocolo](#)
12. [Fase 8 — Frontend .env](#)
13. [Fase 9 — Documentação e Testes](#)
14. [Track Changes por Role](#)
15. [Estrutura de Pastas no Storage](#)
16. [Docker Compose](#)
17. [Ordem de Implementação](#)
18. [Validação / Testes](#)
19. [Código Fonte Completo](#)

1. Visão Geral da Arquitectura



Princípios Fundamentais

1. O frontend nunca lida directamente com o **ONLYOFFICE Document Server**.
2. O frontend pede configuração ao backend via API autenticada.
3. O backend gera um JWT assinado com a configuração do documento e modo de edição.
4. O ONLYOFFICE DS valida o JWT, busca o ficheiro, renderiza o editor.

5. Quando o utilizador salva, o ONLYOFFICE DS chama o **callback do backend**.
 6. O **backend baixa o .docx editado** da URL fornecida pelo ONLYOFFICE e guarda no storage local.
 7. O frontend recarrega a lista de documentos e exhibe o URL público do ficheiro.
-

2. Estado Actual

2.1 O que existe hoje (apenas teste/POC)

Componente	Estado
<code>OnlyOfficeController::config()</code>	Gera config com ficheiro <code>teste.docx</code> (hardcoded, não existe)
<code>OnlyOfficeController::callback()</code>	Stub — só faz <code>Log::info()</code> , não guarda nada
<code>OnlyOfficeEditor</code> (React)	<code>documentServerUrl="http://localhost"</code> hardcoded
Rota de teste <code>/teste-office</code>	Funcional, mas sem documento real
<code>@onlyoffice/document-editor-react</code>	Instalado (v2.2.0)
<code>firebase/php-jwt</code>	Instalado (v7.1)

2.2 Problemas conhecidos

1. **Ficheiro** `teste.docx` **não existe** no storage.
 2. **documentServerUrl hardcoded** — deveria ser `http://localhost:8088` (DS).
 3. **Nenhum documento real** de protocolo está ligado ao editor.
 4. **Callback não guarda** o documento editado.
 5. **0 configuração de track changes** / revisão.
 6. **Rota pública** sem autenticação.
-

3. Fluxo Correcto (Backend Baixa o Documento)

3.1 Abertura do Editor



UTILIZADOR EDITA	
Track changes activado	
Comentários, revisões	

3.2 Salvamento (Callback)

ONLYOFFICE DS Backend (callback)

POST /api/protocolo/onlyoffice	
/callback	
{	
status: 1,	
url: "http://onlyoffice/...",	
key: "protocol_42_v3",	
users: [...],	
changeshistory: [...]	
}	
→	
1. Verifica se status == 1	
2. Extraí protocol_id e	
versão da key	
"protocol_42_v3"	
→ protocol_id=42, v=3	
3. BAIXA .docx da URL:	
file_get_contents(url)	
4. Guarda no storage:	
storage/app/public/	
protocols/42/	
protocol-42-S3.docx	
5. Marca versão anterior	
como "inactive"	
6. Cria registo Document:	
version=3, status=active	
7. Guarda histórico track	
changes (opcional)	

```
|  
| 8. Devolve { error: 0 } |  
| ← { error: 0 } → |
```

4. Ficheiros a Alterar / Criar

Fase 1 — Configuração e Ambiente

`src/backend/.env` (Adicionar)

```
ONLYOFFICE_JWT_SECRET=iscisa_secret_2026453456534563563463546545635463456345643564356  
ONLYOFFICE_DOCUMENT_URL=http://host.docker.internal:8000  
DOCUMENT_SERVER_URL=http://localhost:8088  
FILESYSTEM_DISK=public
```

Variável	Descrição
<code>ONLYOFFICE_JWT_SECRET</code>	Segredo partilhado entre Laravel e ONLYOFFICE DS para JWT
<code>ONLYOFFICE_DOCUMENT_URL</code>	URL pública do Laravel acessível pelo ONLYOFFICE DS (Docker → host)
<code>DOCUMENT_SERVER_URL</code>	URL do ONLYOFFICE Document Server para o frontend
<code>FILESYSTEM_DISK</code>	Disco de filesystem para documentos (<code>public</code>)

Fase 2 — Callback Real

`src/backend/Modules/Protocol/app/Http/Controllers/OnlyOfficeController.php` (Substituir)

```
<?php  
  
namespace Modules\Protocol\app\Http\Controllers;  
  
use App\Http\Controllers\Controller;
```

```

use Firebase\JWT\JWT;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Storage;
use Modules\Protocol\app\Models\Document;
use Modules\Protocol\app\Models\Protocol;
use Modules\User\app\Models\User;

class OnlyOfficeController extends Controller
{
    /**
     * Gera config para um protocolo específico
     * (autenticado – usado pelo frontend)
     */
    public function configForProtocol(int $protocolId)
    {
        $protocol = Protocol::with('latestDocument', 'topic.course')->findOrFail($protocolId);

        $document = $protocol->latestDocument;
        if (!$document) {
            return response()->json(['error' => 'Nenhum documento encontrado'], 404);
        }

        $version = $document->version;
        $key = "protocol_{$protocolId}_v{$version}";
        $user = auth()->user();

        // Determinar modo baseado na role do utilizador
        $mode = $this->resolveMode($user, $protocol);

        $config = [
            "documentType" => "word",
            "document" => [
                "title" => $document->file_name,
                "fileType" => "docx",
                "key" => $key,
                "url" => url("storage/{$document->file_path}"),
            ],
            "editorConfig" => [
                "callbackUrl" => env('ONLYOFFICE_DOCUMENT_URL')
                    . "/api/protocolo/onlyoffice/callback",
                "mode" => $mode,
                "customization" => [
                    "review" => [
                        "reviewMode" => ($mode === 'edit'),

```

```

        "showReviewChanges" => ($mode === 'review' || $mode ===
'edit'),

        "trackChanges" => true,
    ],
    "commentAuthorOnly" => false,
    "forcesave" => true,
],
"user" => [
    "id" => (string) $user->id,
    "name" => $user->name,
],
],
];

$token = JWT::encode(
    $config,
    env('ONLYOFFICE_JWT_SECRET'),
    'HS256'
);

return response()->json([
    'config' => $config,
    'token' => $token,
]);
}

/**
 * Resolve o modo de edição com base no perfil do utilizador
 */
private function resolveMode(User $user, Protocol $protocol): string
{
    // Estudante dono do protocolo → pode editar
    if ((int) $user->id === (int) $protocol->student) {
        return 'edit';
    }

    // Supervisor → modo revisão (vê track changes, aceita/rejeita)
    if ($user->teacherProfile && (int) $user->teacherProfile->id === (int)
$protocol->supervisor_id) {
        return 'review';
    }

    // Revisor → modo comentário (não edita)
    $isReviewer = $protocol->reviewAssignments()
        ->where(function ($q) use ($user) {
            $q->where('reviewer_one', $user->teacherProfile->id)

```



```

        ->orWhere('reviewer_two', $user->teacherProfile?->id);
    })
    ->exists();

    if ($isReviewer) {
        return 'comment';
    }

    // Secretário / Coordenador / Admin → só visualizar
    return 'view';
}

/**
 * Callback chamado pelo ONLYOFFICE Document Server
 * quando o utilizador salva o documento.
 * (público – validado por JWT)
 */
public function callback(Request $request)
{
    $data = $request->all();

    Log::info('ONLYOFFICE callback:', [
        'status' => $data['status'] ?? 'unknown',
        'key' => $data['key'] ?? 'unknown',
    ]);

    // Status ONLYOFFICE:
    // 0 = sem alterações
    // 1 = pronto para guardar (status=1 + url do doc)
    // 2 = a guardar
    // 3 = erro ao guardar
    // 4 = fechado sem alterações
    // 6 = forçar salvar
    // 7 = erro ao forçar salvar

    // Só processamos status=1 (pronto para guardar) e status=6 (forçar salvar)
    $status = (int) ($data['status'] ?? 0);
    if (!in_array($status, [1, 6], true)) {
        return response()->json(['error' => 0]);
    }

    // Extrair protocol_id e versão da key (ex: "protocol_42_v3")
    $key = $data['key'] ?? '';
    preg_match('/protocol_(\d+)_v(\d+)/', $key, $matches);

    if (empty($matches)) {

```

```

        Log::warning('ONLYOFFICE callback: key inválida', ['key' => $key]);
        return response()->json(['error' => 1]);
    }

    $protocolId = (int) $matches[1];
    $version = (int) $matches[2];
    $downloadUrl = $data['url'] ?? null;

    if (!$downloadUrl) {
        Log::warning('ONLYOFFICE callback: URL do documento não fornecida');
        return response()->json(['error' => 1]);
    }

    try {
        // 1. Baixar .docx do ONLYOFFICE
        $docContent = file_get_contents($downloadUrl);

        if ($docContent === false) {
            throw new \Exception("Não foi possível baixar o documento de:
$downloadUrl");
        }

        // 2. Definir caminho e nome do ficheiro
        $fileName = "protocol-{$protocolId}-S{$version}.docx";
        $filePath = "protocols/{$protocolId}/{$fileName}";

        // 3. Guardar no storage público
        Storage::disk('public')->put($filePath, $docContent);

        // 4. Marcar versão anterior como inactive
        Document::where('protocol_id', $protocolId)
            ->where('status', Document::STATUS_ACTIVE)
            ->update(['status' => Document::STATUS_INACTIVE]);

        // 5. Criar nova versão do documento
        Document::create([
            'submitted_by' => $data['user_id'] ?? auth()->id() ?? 1,
            'protocol_id' => $protocolId,
            'document_type' => 'protocol',
            'file_name' => $fileName,
            'file_path' => $filePath,
            'version' => $version,
            'status' => Document::STATUS_ACTIVE,
        ]);

        // 6. Guardar histórico de track changes (opcional)
    }

```

```

        if (!empty($data['changeloghistory'])) {
            Storage::disk('public')->put(
                "protocols/{$protocolId}/history/history-v{$version}.json",
                json_encode($data['changeloghistory'])
            );
        }

        Log::info('ONLYOFFICE callback: documento salvo com sucesso', [
            'protocol_id' => $protocolId,
            'version' => $version,
            'path' => $filePath,
        ]);

        return response()->json(['error' => 0]);
    } catch (\Exception $e) {
        Log::error('ONLYOFFICE callback: erro ao salvar', [
            'error' => $e->getMessage(),
            'protocol_id' => $protocolId,
            'version' => $version,
        ]);

        return response()->json(['error' => 1]);
    }
}
}
}

```

Fase 3 — Config Endpoint para Documentos Reais

`src/backend/Modules/Protocol/app/Http/Controllers/OnlyOfficeController.php`

(Implementado na Fase 2 — métodos `configForProtocol()` e `resolveMode()`)

Fase 4 — Nova Rota Autenticada

`src/backend/Modules/Protocol/routes/api.php`

```

<?php

use Illuminate\Support\Facades\Route;
use Modules\Protocol\app\Http\Controllers\TopicController;

```

```

use Modules\Protocol\app\Http\Controllers\EvaluationFormController;
use Modules\Protocol\app\Http\Controllers\OnlyOfficeController;

// — Rotas PÚBLICAS (só o ONLYOFFICE DS chama) —————
Route::prefix('api')->middleware(['api'])->group(function () {
    Route::get('/onlyoffice/config',
        [OnlyOfficeController::class, 'config']
    );
    Route::post('/protocolo/onlyoffice/callback',
        [OnlyOfficeController::class, 'callback']
    );
});

// — Rotas AUTENTICADAS —————
Route::prefix('api')->middleware(['api', 'auth:sanctum'])->group(function () {
    // Config ONLYOFFICE para um protocolo específico
    Route::get('/onlyoffice/config/{protocol}',
        [OnlyOfficeController::class, 'configForProtocol']
    )->name('onlyoffice.config.protocol');
});

// — Rotas v1 (já existentes) —————
Route::prefix('api/v1')->middleware(['api', 'auth:sanctum'])->group(function () {
    // ... rotas existentes (topics, protocols, evaluations, etc.)
});

```

Fase 5 — Componente Editor React

src/frontend/src/components/OnlyOfficeEditor/00Editor.tsx (Novo)

```

import { useEffect, useState } from 'react'
import { DocumentEditor } from '@onlyoffice/document-editor-react'
import { onlyOfficeService } from '../../../services/onlyOfficeService'

interface Props {
    protocolId: number
    onClose?: () => void
}

export default function 00Editor({ protocolId, onClose }: Props) {
    const [config, setConfig] = useState<any>(null)
    const [loading, setLoading] = useState(true)
    const [error, setError] = useState<string | null>(null)

```

```

const dsUrl = import.meta.env.VITE_DOCUMENT_SERVER_URL || 'http://localhost:8088'

useEffect(() => {
  let cancelled = false

  async function load() {
    setLoading(true)
    setError(null)
    try {
      const response = await onlyOfficeService.getConfig(protocolId)
      if (cancelled) return

      setConfig({
        ...response.config,
        token: response.token,
      })
    } catch (e) {
      if (!cancelled) setError((e as Error).message)
    } finally {
      if (!cancelled) setLoading(false)
    }
  }

  load()
  return () => { cancelled = true }
}, [protocolId])

if (loading) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',
    height: '100%', fontFamily: 'var(--font-family)',
    color: 'var(--on-surface-variant)', fontSize: 'var(--body-lg)', gap: 8
  }}>
    <span style={{
      width: 24, height: 24, border: '3px solid var(--outline-variant)',
      borderTopColor: 'var(--primary)', borderRadius: '50%',
      animation: 'spin 0.8s linear infinite'
    }} />
    A carregar editor...
  </div>
)

if (error) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',

```

```

        height: '100%', color: 'var(--error)', fontSize: 'var(--body-lg)'
    }}>
    Erro: {error}
</div>
)

if (!config) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',
    height: '100%', color: 'var(--on-surface-variant)', fontSize: 'var(--body-lg)'
  }}>
    Não foi possível carregar o editor.
  </div>
)

return (
  <div style={{ position: 'relative', height: '100%', width: '100%' }}>
    {onClose && (
      <button
        onClick={onClose}
        style={{
          position: 'absolute', top: 8, right: 8, zIndex: 100,
          padding: '6px 12px', borderRadius: 6, border: '1px solid var(--outline)',
          background: 'var(--surface-container)', cursor: 'pointer',
          fontSize: 'var(--body-md)', display: 'flex', alignItems: 'center', gap: 4
        }}
      >
        <span className="material-symbols-outlined" style={{ fontSize: 18
}}>close</span>
        Fechar
      </button>
    )}
    <DocumentEditor
      id="onlyoffice-editor"
      documentServerUrl={dsUrl}
      config={config}
      height="100%"
      width="100%"
    />
  </div>
)
}

```

`src/frontend/src/components/OnlyOfficeEditor/index.ts` (Novo)

```
export { default } from './OOEditor'
```

Fase 6 — Serviço Frontend Atualizado

`src/frontend/src/services/onlyOfficeService.ts`

```
const BASE = import.meta.env.VITE_API_BASE_URL ?? 'http://localhost:8000/api'

function token(): string | null {
  return localStorage.getItem('sgpmc_token')
}

export interface OnlyOfficeConfigResponse {
  config: any
  token: string
}

export const onlyOfficeService = {
  async getConfig(protocolId?: number): Promise<OnlyOfficeConfigResponse> {
    const headers: Record<string, string> = {
      Accept: 'application/json',
    }

    const authToken = token()
    if (authToken) {
      headers['Authorization'] = `Bearer ${authToken}`
    }

    const endpoint = protocolId
      ? `${BASE}/onlyoffice/config/${protocolId}`
      : `${BASE}/onlyoffice/config`

    const res = await fetch(endpoint, { method: 'GET', headers })
    const data = await res.json()

    if (!res.ok) {
      throw new Error(data.message ?? 'Erro ao carregar configuração do ONLYOFFICE')
    }

    return data
  },
}
```

```
}
```

Fase 7 — Integrar Editor nas Páginas de Protocolo

`src/frontend/src/pages/student/ProtocolPage.tsx` (Adições)

Junto aos documentos activos, adicionar botão "Editar Online":

```
// Dentro do card de cada protocolo, após a secção de documentos:
{activeDocs.length > 0 && (
  <div style={{
    display: 'flex', gap: 'var(--space-1)',
    marginTop: 'var(--space-2)', flexWrap: 'wrap'
  }}>
    <a
      href={doc.file_url}
      target="_blank"
      rel="noreferrer"
      className="btn btn-outline"
    >
      <span className="material-symbols-outlined">download</span>
      Descarregar
    </a>

    {[ 'protocol_pending_nucleo', 'protocol_in_review_nucleo',
      'protocol_pending_comite_cientifico',
      'protocol_pending_comite_bioetica' ].includes(p.status) && (
      <button
        onClick={() => setEditingProtocolId(p.id)}
        className="btn btn-primary"
      >
        <span className="material-symbols-outlined">edit</span>
        Editar Online
      </button>
    )}
  </div>
)}
```

Modal/Overlay para o editor:

```
import OOEEditor from '../components/OnlyOfficeEditor'
```



```

export default function ProtocolPage() {
  const [editingProtocolId, setEditingProtocolId] = useState<number | null>(null)

  // ... resto do componente

  return (
    <div>
      {/* ... conteúdo existente ... */}

      {/* Modal do ONLYOFFICE */}
      {editingProtocolId && (
        <div
          style={{
            position: 'fixed', inset: 0, zIndex: 9999,
            background: 'rgba(0,0,0,0.6)',
            display: 'flex', alignItems: 'center', justifyContent: 'center',
          }}
          onClick={() => setEditingProtocolId(null)}
        >
          <div
            style={{
              width: '95vw', height: '95vh',
              background: '#fff', borderRadius: 8, overflow: 'hidden',
              display: 'flex', flexDirection: 'column'
            }}
            onClick={e => e.stopPropagation()}
          >
            <OOEditor
              protocolId={editingProtocolId}
              onClose={() => {
                setEditingProtocolId(null)
                load() // recarregar lista de protocolos
              }}
            />
          </div>
        </div>
      )}
    </div>
  )
}

```

src/frontend/src/pages/student/DocumentsPage.tsx (Adições)

```

// Na tabela de documentos, adicionar coluna/botão "Editar"
<td style={{ padding: 'var(--space-2) var(--space-4)' }}>

```

```

<button
  onClick={() => setEditingProtocolId(p.id)}
  className="btn btn-sm btn-primary"
  title="Editar online"
>
  <span className="material-symbols-outlined" style={{ fontSize: 16 }}>edit</span>
</button>
</td>

```

Fase 8 — Frontend .env

src/frontend/.env

```

VITE_API_BASE_URL=http://localhost:8000/api
VITE_DOCUMENT_SERVER_URL=http://localhost:8088

```

Fase 9 — Documentação e Testes

docs/ONLYOFFICE_INTEGRATION.md (Este documento)

docs/ONLYOFFICE_TROUBLESHOOTING.md (Recomendado)

Conteúdo sugerido: - Verificar conectividade Docker → Laravel - Logs do ONLYOFFICE DS: `docker logs onlyoffice-document-server` - Logs do Laravel: `storage/logs/laravel.log` - Verificar JWT secret 一致 - Verificar permissões de storage

5. Track Changes por Role

O ONLYOFFICE Document Server suporta **track changes nativamente**. Configuramos por role:

Role	editorConfig.mod e	review.reviewMod e	review.trackChange s	Permissões
Estudante	edit	true	true	Edita,

Role	editorConfig.mod e	review.reviewMod e	review.trackChange s	Permissões
(dono)				aceita/rejeita alterações
Supervisor	review	true	true	Vê alterações, aceita/rejeita
Revisor	comment	false	false	Só comentários, sem editar
Secretário	view	false	false	Só visualizar
Coordenador	view	false	false	Só visualizar
Admin	edit	true	true	Edita, aceita/rejeita

Como o ONLYOFFICE lida com Track Changes

1. Quando `trackChanges: true` e `mode: 'edit'`, o editor activa o modo "controlo de alterações":
2. Inserções ficam sublinhadas a verde
3. Remoções ficam riscadas a vermelho
4. Barra lateral de revisão mostra quem alterou o quê
5. Utilizadores em modo `review` podem:
6. Ver todas as alterações destacadas
7. Aceitar ou rejeitar cada alteração
8. Adicionar comentários
9. Utilizadores em modo `comment` só podem adicionar comentários.

Histórico de Alterações

O ONLYOFFICE DS envia no callback: - `changeshistory`: array com histórico completo de alterações - `changeshistoryurl`: URL para baixar o histórico em formato JSON

O backend guarda este histórico em:

```
storage/app/public/protocols/{id}/history/history-v{version}.json
```

6. Estrutura de Pastas no Storage

```
storage/app/public/
├── protocols/
│   ├── 42/
│   │   ├── protocol-42-S1.docx      ← Versão 1 (upload inicial)
│   │   ├── protocol-42-S2.docx      ← Versão 2 (editado via 00)
│   │   ├── protocol-42-S3.docx      ← Versão 3 (editado via 00)
│   │   └── history/
│   │       ├── history-v2.json       ← Track changes da versão 2
│   │       └── history-v3.json       ← Track changes da versão 3
│   └── 43/
│       ├── protocol-43-S1.docx
│       └── ...
├── monographs/
│   └── ...
└── documents/
    └── teste.docx                    ← Ficheiro de teste (actual)
```

7. Docker Compose

`docker-compose.yml` (para o ONLYOFFICE Document Server)

```
version: '3'

services:
  onlyoffice-document-server:
    image: onlyoffice/documentserver:latest
    container_name: onlyoffice-ds
```

```
ports:
  - "8088:80"
environment:
  - JWT_ENABLED=true
  - JWT_SECRET=iscisa_secret_2026453456534563563463546545635463456345643564356
  - WOPI_ENABLED=false
volumes:
  - onlyoffice_data:/var/www/onlyoffice/Data
  - onlyoffice_log:/var/log/onlyoffice
restart: unless-stopped

volumes:
  onlyoffice_data:
  onlyoffice_log:
```

Rede Docker

Para o ONLYOFFICE DS conseguir aceder ao Laravel (buscar ficheiros e enviar callback), ambos precisam de estar na mesma rede Docker ou o Laravel precisa de estar acessível via `host.docker.internal`.

Se o Laravel estiver a correr localmente (não em Docker):

```
# No docker-compose, adicionar:
extra_hosts:
  - "host.docker.internal:host-gateway"
```

Verificação da Instalação

```
# Verificar se o ONLYOFFICE DS está a correr
docker ps | grep onlyoffice

# Aceder à página de verificação
curl http://localhost:8088/healthcheck

# Ver logs do ONLYOFFICE DS
docker logs onlyoffice-ds --tail 50
```

8. Ordem de Implementação

Fase	Tarefa	Ficheiros	Esforço
1	Configuração .env	.env (backend + frontend)	10 min
2	Callback real	onlyOfficeController.php	2h
3	Config endpoint por protocolo	onlyOfficeController.php	2h
4	Rota autenticada	routes/api.php	10 min
5	Componente React	00Editor.tsx, index.ts	1h
6	Serviço actualizado	onlyOfficeService.ts	15 min
7	Botões "Editar" nas páginas	ProtocolPage.tsx, DocumentsPage.tsx	2h
8	Docker Compose + testes	docker-compose.yml	1h
9	Documentação	docs/ONLYOFFICE_INTEGRATION.md	30 min
Total		~12 ficheiros	~9 horas

Dependências

Fase 1 → Fase 2 → Fase 3 → Fase 4 → Fase 5 → Fase 7

↑
Fase 6

Fase 8 (paralelo)

Fase 9 (final)

9. Validação / Testes

Teste 1 — Abertura do Editor com Documento Real

Pré-condições: - Estudante com tema aprovado - Protocolo submetido (pelo menos 1 versão)

Passos: 1. Login como estudante 2. Navegar para "O meu protocolo" 3. Verificar que o documento aparece na lista 4. Clicar "Editar Online" 5. **Esperado:** Editor ONLYOFFICE abre com o documento do protocolo

Teste 2 — Salvamento via Callback

Passos: 1. No editor, fazer alterações no texto 2. Clicar "Salvar" (ou Ctrl+S) 3. Fechar o editor 4. Recarregar a página 5. **Esperado:** Nova versão do documento aparece na lista 6. Clicar para descarregar 7. **Esperado:** O ficheiro contém as alterações feitas

Teste 3 — Track Changes para Supervisor

Passos: 1. Estudante edita e salva o documento 2. Login como supervisor 3. Abrir o mesmo protocolo 4. Clicar "Editar Online" 5. **Esperado:** Editor abre em modo "review" com alterações destacadas 6. Supervisor pode aceitar/rejeitar alterações

Teste 4 — Modo Comentário para Revisor

Passos: 1. Login como revisor 2. Aceder a revisão atribuída 3. Clicar "Editar Online" 4. **Esperado:** Editor abre em modo "comment" 5. Revisor não consegue editar o texto, só adicionar comentários

Teste 5 — Modo Visualização para Secretário

Passos: 1. Login como secretário 2. Aceder a um protocolo 3. Clicar "Visualizar" 4. **Esperado:** Editor abre em modo "view" 5. Secretário não consegue editar nem comentar

Teste 6 — Versionamento

Passos: 1. Submeter protocolo (v1) 2. Editar e salvar (v2) 3. Editar e salvar novamente (v3) 4. **Esperado:** 3 versões na tabela `documents` 5. Apenas a v3 está `active` 6. v1 e v2 estão `inactive`

10. Código Fonte Completo

10.1 OnlyOfficeController.php (completo)

```
<?php

namespace Modules\Protocol\app\Http\Controllers;

use App\Http\Controllers\Controller;
use Firebase\JWT\JWT;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\Facades\Storage;
use Modules\Protocol\app\Models\Document;
use Modules\Protocol\app\Models\Protocol;
use Modules\User\app\Models\User;

class OnlyOfficeController extends Controller
{
    /**
     * Config genérica para teste (existente)
     */
    public function config()
    {
        $config = [
            "documentType" => "word",
            "document" => [
                "title" => "teste.docx",
                "fileType" => "docx",
                "key" => "teste_001",
                "url" => env('ONLYOFFICE_DOCUMENT_URL') .
                "/storage/documents/teste.docx",
            ],
            "editorConfig" => [
                "callbackUrl" => env('ONLYOFFICE_DOCUMENT_URL')
                . "/api/protocolo/onlyoffice/callback",
            ],
        ];
    }
}
```



```

    ],
  ];

  $token = JWT::encode(
    $config,
    env('ONLYOFFICE_JWT_SECRET'),
    'HS256'
  );

  return response()->json([
    'config' => $config,
    'token' => $token,
  ]);
}

/**
 * Config para um protocolo específico (autenticado)
 */
public function configForProtocol(int $protocolId)
{
    $protocol = Protocol::with('latestDocument', 'topic.course')->findOrFail($protocolId);

    $document = $protocol->latestDocument;
    if (!$document) {
        return response()->json(['error' => 'Nenhum documento encontrado'], 404);
    }

    $version = $document->version;
    $key = "protocol_{$protocolId}_v{$version}";
    $user = auth()->user();
    $mode = $this->resolveMode($user, $protocol);

    $config = [
        "documentType" => "word",
        "document" => [
            "title" => $document->file_name,
            "fileType" => "docx",
            "key" => $key,
            "url" => url("storage/{$document->file_path}"),
        ],
        "editorConfig" => [
            "callbackUrl" => env('ONLYOFFICE_DOCUMENT_URL')
                . "/api/protocolo/onlyoffice/callback",
            "mode" => $mode,
            "customization" => [

```

```

        "review" => [
            "reviewMode" => ($mode === 'edit'),
            "showReviewChanges" => ($mode === 'review' || $mode ===
'edit'),

            "trackChanges" => true,
        ],
        "commentAuthorOnly" => false,
        "forcesave" => true,
    ],
    "user" => [
        "id" => (string) $user->id,
        "name" => $user->name,
    ],
],
];

$token = JWT::encode(
    $config,
    env('ONLYOFFICE_JWT_SECRET'),
    'HS256'
);

return response()->json([
    'config' => $config,
    'token' => $token,
]);
}

/**
 * Resolve o modo de edição baseado na role
 */
private function resolveMode(User $user, Protocol $protocol): string
{
    if ((int) $user->id === (int) $protocol->student) {
        return 'edit';
    }

    if ($user->teacherProfile && (int) $user->teacherProfile->id === (int)
$protocol->supervisor_id) {
        return 'review';
    }

    $isReviewer = $protocol->reviewAssignments()
        ->where(function ($q) use ($user) {
            $q->where('reviewer_one', $user->teacherProfile->id)
                ->orWhere('reviewer_two', $user->teacherProfile->id);
        });

```

```

    })
    ->exists();

    if ($isReviewer) {
        return 'comment';
    }

    return 'view';
}

/**
 * Callback do ONLYOFFICE DS (público)
 */
public function callback(Request $request)
{
    $data = $request->all();
    Log::info('ONLYOFFICE callback:', [
        'status' => $data['status'] ?? 'unknown',
        'key' => $data['key'] ?? 'unknown',
    ]);

    $status = (int) ($data['status'] ?? 0);
    if (!in_array($status, [1, 6], true)) {
        return response()->json(['error' => 0]);
    }

    $key = $data['key'] ?? '';
    preg_match('/protocol_(\d+)_v(\d+)/', $key, $matches);

    if (empty($matches)) {
        Log::warning('ONLYOFFICE callback: key inválida');
        return response()->json(['error' => 1]);
    }

    $protocolId = (int) $matches[1];
    $version = (int) $matches[2];
    $downloadUrl = $data['url'] ?? null;

    if (!$downloadUrl) {
        Log::warning('ONLYOFFICE callback: URL não fornecida');
        return response()->json(['error' => 1]);
    }

    try {
        $docContent = file_get_contents($downloadUrl);
        if ($docContent === false) {

```

```

        throw new \Exception("Falha ao baixar: $downloadUrl");
    }

    $fileName = "protocol-{$protocolId}-S{$version}.docx";
    $filePath = "protocols/{$protocolId}/{$fileName}";
    Storage::disk('public')->put($filePath, $docContent);

    Document::where('protocol_id', $protocolId)
        ->where('status', Document::STATUS_ACTIVE)
        ->update(['status' => Document::STATUS_INACTIVE]);

    Document::create([
        'submitted_by' => $data['user_id'] ?? auth()->id() ?? 1,
        'protocol_id' => $protocolId,
        'document_type' => 'protocol',
        'file_name' => $fileName,
        'file_path' => $filePath,
        'version' => $version,
        'status' => Document::STATUS_ACTIVE,
    ]);

    if (!empty($data['changeloghistory'])) {
        Storage::disk('public')->put(
            "protocols/{$protocolId}/history/history-v{$version}.json",
            json_encode($data['changeloghistory'])
        );
    }

    Log::info('ONLYOFFICE callback: salvo com sucesso', [
        'protocol_id' => $protocolId,
        'version' => $version,
    ]);

    return response()->json(['error' => 0]);
} catch (\Exception $e) {
    Log::error('ONLYOFFICE callback: erro', [
        'error' => $e->getMessage(),
        'protocol_id' => $protocolId,
    ]);

    return response()->json(['error' => 1]);
}
}
}

```

10.2 routes/api.php

```
<?php

use Illuminate\Support\Facades\Route;
use Modules\Protocol\app\Http\Controllers\TopicController;
use Modules\Protocol\app\Http\Controllers\EvaluationFormController;
use Modules\Protocol\app\Http\Controllers\OnlyOfficeController;

// Rotas públicas (ONLYOFFICE DS)
Route::prefix('api')->middleware(['api'])->group(function () {
    Route::get('/onlyoffice/config',
        [OnlyOfficeController::class, 'config']
    );
    Route::post('/protocolo/onlyoffice/callback',
        [OnlyOfficeController::class, 'callback']
    );
});

// Rotas autenticadas
Route::prefix('api')->middleware(['api', 'auth:sanctum'])->group(function () {
    Route::get('/onlyoffice/config/{protocol}',
        [OnlyOfficeController::class, 'configForProtocol']
    )->name('onlyoffice.config.protocol');
});

// Rotas v1 (existentes)
Route::prefix('api/v1')->middleware(['api', 'auth:sanctum'])->group(function () {
    Route::get('/topics/my-approved',
        [TopicController::class, 'getMyApprovedTopics']
    );
    Route::post('topics', [TopicController::class, 'store']);
    Route::get('topics', [TopicController::class, 'index']);
    Route::patch('topics/{topic}/supervisor-approve',
        [TopicController::class, 'approveBySupervisor']
    );
    Route::patch('topics/{topic}/supervisor-reject',
        [TopicController::class, 'rejectBySupervisor']
    );
    Route::get('supervisor/topics',
        [TopicController::class, 'getForSupervisor']
    );
    Route::get('secretary/topics',
        [TopicController::class, 'getForSecretary']
    );
    Route::get('topics/{topic}/eligible-reviewers',
        [TopicController::class, 'getEligibleReviewers']
    );
});
```

```

);
Route::post('topics/{topic}/assign-reviewers',
    [TopicController::class, 'assignReviewers']
);
Route::get('reviewer/topics',
    [TopicController::class, 'getForReviewer']
);
Route::get('topics/{topic}/comments',
    [TopicController::class, 'getComments']
);
Route::post('topics/{topic}/comments',
    [TopicController::class, 'submitComment']
);
Route::post('topics/{topic}/evaluations',
    [TopicController::class, 'submitEvaluation']
);

Route::get('supervisor/protocols',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@getForSupervisor'
);
Route::post('protocols',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@store'
);
Route::get('protocols',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@index'
);
Route::get('protocols/{protocol}',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@show'
);
Route::patch('protocols/{protocol}/supervisor-approve',
    'Modules\Protocol\app\Http\Controllers\
ProtocolApiController@approveBySupervisor'
);
Route::patch('protocols/{protocol}/supervisor-reject',
    'Modules\Protocol\app\Http\Controllers\
ProtocolApiController@rejectBySupervisor'
);
Route::get('secretary/protocols',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@getForSecretary'
);
Route::get('protocols/{protocol}/eligible-reviewers',
    'Modules\Protocol\app\Http\Controllers\
ProtocolApiController@getEligibleReviewers'
);
Route::post('protocols/{protocol}/assign-reviewers',
    'Modules\Protocol\app\Http\Controllers\ProtocolApiController@assignReviewers'

```

```

    );
    Route::get('reviewer/protocols',
        'Modules\Protocol\app\Http\Controllers\ProtocolApiController@getForReviewer'
    );
    Route::get('evaluation-forms/{form}',
        [EvaluationFormController::class, 'show']
    );
    Route::post('evaluation-forms/{form}/criteria/{formCriterion}/review',
        [EvaluationFormController::class, 'saveCriterionReview']
    );
    Route::post('evaluation-forms/{form}/submit',
        [EvaluationFormController::class, 'submit']
    );
    Route::post('evaluation-forms/{form}/decide',
        [EvaluationFormController::class, 'decide']
    );
    Route::get('reviewer/evaluations',
        [EvaluationFormController::class, 'getForReviewer']
    );
    Route::get('secretary/evaluations',
        [EvaluationFormController::class, 'getForSecretary']
    );
    Route::get('opinions/{opinion}/download',
        [EvaluationFormController::class, 'downloadOpinion']
    );
    Route::get('protocols/{protocol}/opinions',
        [EvaluationFormController::class, 'listOpinionsForProtocol']
    );
});

```

10.3 OOEEditor.tsx (componente React completo)

```

import { useEffect, useState } from 'react'
import { DocumentEditor } from '@onlyoffice/document-editor-react'
import { onlyOfficeService } from '../../services/onlyOfficeService'

interface Props {
    protocolId: number
    onClose?: () => void
}

export default function OOEEditor({ protocolId, onClose }: Props) {
    const [config, setConfig] = useState<any>(null)
    const [loading, setLoading] = useState(true)
    const [error, setError] = useState<string | null>(null)

```

```

const dsUrl = import.meta.env.VITE_DOCUMENT_SERVER_URL || 'http://localhost:8088'

useEffect(() => {
  let cancelled = false

  async function load() {
    setLoading(true)
    setError(null)
    try {
      const response = await onlyOfficeService.getConfig(protocolId)
      if (cancelled) return

      setConfig({
        ...response.config,
        token: response.token,
      })
    } catch (e) {
      if (!cancelled) setError((e as Error).message)
    } finally {
      if (!cancelled) setLoading(false)
    }
  }

  load()
  return () => { cancelled = true }
}, [protocolId])

if (loading) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',
    height: '100%', fontFamily: 'var(--font-family)',
    color: 'var(--on-surface-variant)', fontSize: 'var(--body-lg)', gap: 8
  }}>
    <span style={{
      width: 24, height: 24, border: '3px solid var(--outline-variant)',
      borderTopColor: 'var(--primary)', borderRadius: '50%',
      animation: 'spin 0.8s linear infinite'
    }} />
    A carregar editor...
  </div>
)

if (error) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',

```



```

        height: '100%', color: 'var(--error)', fontSize: 'var(--body-lg)'
    }}>
    Erro: {error}
</div>
)

if (!config) return (
  <div style={{
    display: 'flex', alignItems: 'center', justifyContent: 'center',
    height: '100%', color: 'var(--on-surface-variant)', fontSize: 'var(--body-lg)'
  }}>
    Não foi possível carregar o editor.
  </div>
)

return (
  <div style={{ position: 'relative', height: '100%', width: '100%' }}>
    {onClose && (
      <button
        onClick={onClose}
        style={{
          position: 'absolute', top: 8, right: 8, zIndex: 100,
          padding: '6px 12px', borderRadius: 6, border: '1px solid var(--outline)',
          background: 'var(--surface-container)', cursor: 'pointer',
          fontSize: 'var(--body-md)', display: 'flex', alignItems: 'center', gap: 4,
          fontFamily: 'var(--font-family)',
        }}
      >
        <span className="material-symbols-outlined" style={{ fontSize: 18
}}>close</span>
        Fechar
      </button>
    )}
    <DocumentEditor
      id="onlyoffice-editor"
      documentServerUrl={dsUrl}
      config={config}
      height="100%"
      width="100%"
    />
  </div>
)
}

```

Apêndice A — Referências

- [ONLYOFFICE Document Server API](#)
 - [ONLYOFFICE JWT Integration](#)
 - [@onlyoffice/document-editor-react](#)
 - [Laravel Storage](#)
 - [firebase/php-jwt](#)
-

Apêndice B — Checklist de Implementação

- [] Variáveis .env configuradas (backend + frontend)
 - [] ONLYOFFICE Document Server em Docker
 - [] `onlyOfficeController::configForProtocol()` implementado
 - [] `onlyOfficeController::callback()` implementado (baixar + guardar)
 - [] `onlyOfficeController::resolveMode()` implementado (role-based)
 - [] Rotas actualizadas no `routes/api.php`
 - [] Componente `00Editor.tsx` criado
 - [] Serviço `onlyOfficeService.ts` actualizado
 - [] Botões "Editar Online" nas páginas de protocolo
 - [] Modal/overlay para o editor
 - [] Recarregar lista após edição
 - [] Track changes configurado por role
 - [] Testes de todas as roles
-